LISTEN / NOTIFY in PostgreSQL

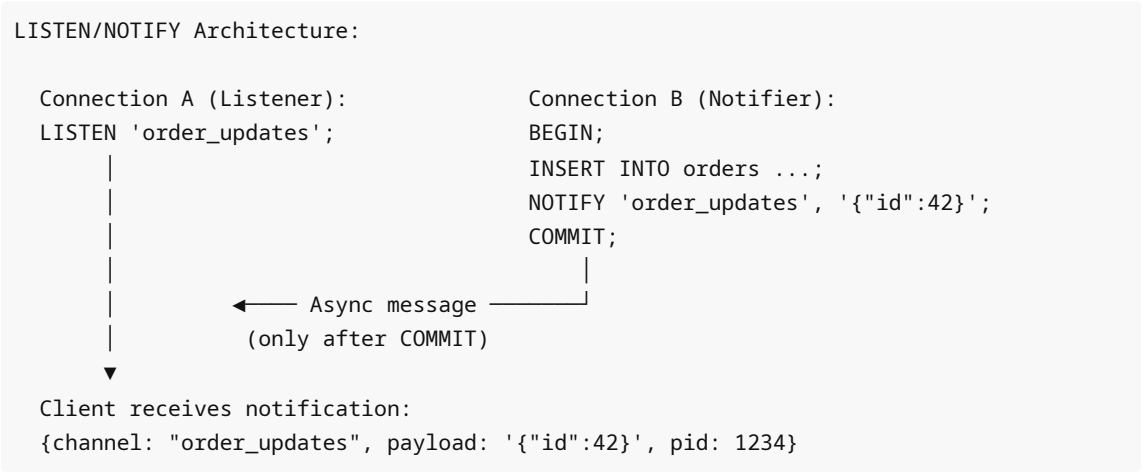
Table of Contents

Learning Objectives

By the end of this module you will be able to:

- Explain how PostgreSQL's async notification system works under the hood
- Implement LISTEN/NOTIFY for real-time event broadcasting
- Use `pg_notify()` from triggers and stored procedures
- Build cache invalidation systems using NOTIFY
- Design event-driven architectures on top of PostgreSQL
- Understand the limitations and failure modes of NOTIFY

How LISTEN/NOTIFY Works



Key properties:

1. **Asynchronous** — listeners receive notifications when they next check (not real-time push)
2. **After COMMIT** — notifications are discarded if the transaction rolls back

3. **PostgreSQL-internal** — no external broker needed
4. **Best-effort delivery** — if the listener is disconnected, the notification is lost
5. **Payload limit** — 8000 bytes max (use IDs, not full data)

LISTEN and UNLISTEN

```
-- Start listening on a channel
LISTEN order_updates;
LISTEN cache_invalidation;
LISTEN 'user.profile.changed';  -- dots allowed, case-insensitive

-- Listen in psql — messages appear asynchronously
-- In terminal 1:
LISTEN test_channel;
-- (now waiting)

-- In terminal 2:
NOTIFY test_channel, 'hello world';
-- Terminal 1 receives:
-- Asynchronous notification "test_channel" with payload "hello world" received from
server process with PID 12345.

-- Stop listening
UNLISTEN order_updates;
UNLISTEN *;  -- stop listening on all channels

-- Check active listeners in the database
SELECT pid, relname AS channel, listen_type
FROM pg_listening_channels()
-- Note: pg_listening_channels() shows channels for CURRENT connection

-- Check all listeners across all connections
SELECT pid, channel
FROM pg_stat_activity, pg_listening_channels() AS channel
-- This doesn't work directly; use the system below:
SELECT * FROM pg_notification_listeners;  -- Not a real view — see monitoring
section
```

NOTIFY and pg_notify

```
-- NOTIFY is a SQL command (outside transactions, immediate delivery after commit)
NOTIFY order_updates;                          -- no payload
NOTIFY order_updates, 'order_id=42';           -- with payload string

-- NOTIFY inside a transaction (delivered AFTER commit)
BEGIN;
    UPDATE orders SET status = 'shipped' WHERE id = 42;
    NOTIFY order_updates, '{"id": 42, "status": "shipped"}';
```

```
COMMIT;  -- listener receives notification here

-- NOTIFY in a rolled-back transaction → notification is DISCARDED
BEGIN;
    UPDATE orders SET status = 'shipped' WHERE id = 42;
    NOTIFY order_updates, '{"id": 42}';
ROLLBACK;  -- notification NOT sent

-- pg_notify() function — same as NOTIFY but usable in SQL/PL/pgSQL
SELECT pg_notify('order_updates', 'order_id=42');

-- Can be used in DO blocks
DO $$
BEGIN
    PERFORM pg_notify('system_alert', 'Database maintenance starting in 5 minutes');
END;
$$;

-- Can be used in functions
CREATE OR REPLACE FUNCTION notify_order_change(p_order_id INT, p_status TEXT)
RETURNS void LANGUAGE plpgsql AS $$
BEGIN
    PERFORM pg_notify(
        'order_updates',
        json_build_object('order_id', p_order_id, 'status', p_status)::text
    );
END;
$$;
```

Payload Support

```
-- Payload is always TEXT, max 8000 bytes
-- Best practices for payload design:

-- 1. Minimal payload: just the ID (let consumer query for full data)
PERFORM pg_notify('user_changed', user_id::text);

-- 2. Structured payload with key fields
PERFORM pg_notify('order_status_changed',
    json_build_object(
        'order_id', NEW.id,
        'old_status', OLD.status,
        'new_status', NEW.status,
        'changed_at', now()
    )::text
);

-- 3. Cache invalidation payload (entity type + key)
PERFORM pg_notify('cache_invalidate',
    json_build_object(
```

```
        'entity',   'product',
        'key',      NEW.id,
        'operation', TG_OP
    )::text
);

-- DO NOT send full row data if it might exceed 8000 bytes
-- Check payload size:
SELECT LENGTH(json_agg(t)::text) FROM large_table t WHERE id = 1;
-- If > 8000 bytes, just send the ID
```

Trigger-based Notifications

```
-- Generic notification trigger
CREATE OR REPLACE FUNCTION table_change_notify()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_payload  TEXT;
    v_id       BIGINT;
BEGIN
    v_id := COALESCE(NEW.id, OLD.id);
    v_payload := json_build_object(
        'table',     TG_TABLE_NAME,
        'operation', TG_OP,
        'id',        v_id
    )::text;

    -- Truncate payload if too long (safety measure)
    IF LENGTH(v_payload) > 7500 THEN
        v_payload := json_build_object(
            'table', TG_TABLE_NAME,
            'op',    TG_OP,
            'id',    v_id
        )::text;
    END IF;

    PERFORM pg_notify('table_changes', v_payload);
    RETURN COALESCE(NEW, OLD);
END;
$$;

-- Attach to tables
CREATE TRIGGER trg_orders_notify
    AFTER INSERT OR UPDATE OR DELETE ON orders
    FOR EACH ROW EXECUTE FUNCTION table_change_notify();

CREATE TRIGGER trg_products_notify
    AFTER INSERT OR UPDATE OR DELETE ON products
    FOR EACH ROW EXECUTE FUNCTION table_change_notify();
```

```
-- Fine-grained: only notify on status change
CREATE OR REPLACE FUNCTION order_status_notify()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF OLD.status IS DISTINCT FROM NEW.status THEN
        PERFORM pg_notify('order_status',
            json_build_object(
                'order_id',   NEW.id,
                'customer_id', NEW.customer_id,
                'old_status', OLD.status,
                'new_status', NEW.status
            )::text
        );
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_order_status_change
    AFTER UPDATE OF status ON orders
    FOR EACH ROW EXECUTE FUNCTION order_status_notify();
```

Real-Time Use Cases

Use Case 1: Cache Invalidation

```
Application Architecture with LISTEN/NOTIFY:

  PostgreSQL                    Application Server(s)
  ┌──────────────────┐          ┌──────────────────────────────┐
  │ UPDATE products  │          │  Server 1:                   │
  │   SET price=...  │          │  LISTEN cache_invalidate     │
  │                  │          │  On notify → del Redis key   │
  │ NOTIFY           │◄─────────┤                              │
  │  'cache_inval'   │─────────►│  Server 2:                   │
  │  'product:42'    │          │  LISTEN cache_invalidate     │
  └──────────────────┘          │  On notify → del Redis key   │
                                └──────────────────────────────┘
```

```
-- Trigger that fires cache invalidation
CREATE OR REPLACE FUNCTION invalidate_product_cache()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    PERFORM pg_notify('cache_invalidate',
        'product:' || COALESCE(NEW.id, OLD.id)::text
    );
    RETURN COALESCE(NEW, OLD);
END;
$$;
```

```
CREATE TRIGGER trg_product_cache_invalidate
    AFTER INSERT OR UPDATE OR DELETE ON products
    FOR EACH ROW EXECUTE FUNCTION invalidate_product_cache();
```

Use Case 2: Real-Time Dashboard Updates

```
-- Dashboard listens for aggregation updates
-- Application does:
-- conn.listen('dashboard_refresh')
-- When notified: call refresh_dashboard_data() and push to WebSocket clients

CREATE OR REPLACE FUNCTION notify_dashboard_refresh()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    PERFORM pg_notify('dashboard_refresh',
        json_build_object(
            'metric', TG_TABLE_NAME,
            'ts',     EXTRACT(EPOCH FROM now())::int
        )::text
    );
    RETURN COALESCE(NEW, OLD);
END;
$$;
```

Use Case 3: Worker Queue Wake-up

```
-- Background worker uses LISTEN to sleep efficiently
-- Instead of polling: SELECT * FROM job_queue WHERE status='pending'
-- Use LISTEN to be woken up when new jobs arrive

CREATE TABLE job_queue (
    id         BIGSERIAL PRIMARY KEY,
    job_type   TEXT NOT NULL,
    payload    JSONB,
    status     TEXT DEFAULT 'pending',
    created_at TIMESTAMPTZ DEFAULT now()
);

CREATE OR REPLACE FUNCTION notify_new_job()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    PERFORM pg_notify('new_job',
        json_build_object('id', NEW.id, 'type', NEW.job_type)::text
    );
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_new_job
```

```
    AFTER INSERT ON job_queue
    FOR EACH ROW EXECUTE FUNCTION notify_new_job();

-- Worker pseudo-code:
-- LISTEN new_job
-- WHILE running:
--   wait for notification (with 30s timeout)
--   SELECT ... FROM job_queue WHERE status='pending' FOR UPDATE SKIP LOCKED LIMIT 1
--   process job
--   UPDATE job_queue SET status='done' WHERE id=?
```

Use Case 4: Schema Change Notifications

```
-- Notify when DDL changes happen (for schema management tooling)
CREATE OR REPLACE FUNCTION notify_schema_change()
RETURNS event_trigger LANGUAGE plpgsql AS $$
DECLARE
    r RECORD;
BEGIN
    FOR r IN SELECT * FROM pg_event_trigger_ddl_commands() LOOP
        PERFORM pg_notify('schema_change',
            json_build_object(
                'command_tag', r.command_tag,
                'object_type', r.object_type,
                'object_identity', r.object_identity
            )::text
        );
    END LOOP;
END;
$$;

CREATE EVENT TRIGGER ddl_change_notify
    ON ddl_command_end
    EXECUTE FUNCTION notify_schema_change();
```

Application Integration Patterns

Python (psycopg2)

```
import psycopg2
import select
import json

conn = psycopg2.connect("host=localhost dbname=mydb user=app password=pass")
conn.set_isolation_level(0)  # autocommit — required for LISTEN

cur = conn.cursor()
cur.execute("LISTEN order_updates;")
cur.execute("LISTEN cache_invalidate;")
```

```
print("Waiting for notifications...")
while True:
    # Wait up to 30 seconds
    if select.select([conn], [], [], 30) == ([], [], []):
        print("Timeout — no notifications")
        continue

    conn.poll()
    while conn.notifies:
        notification = conn.notifies.pop(0)
        payload = json.loads(notification.payload) if notification.payload else {}
        print(f"Channel: {notification.channel}, Payload: {payload}")

        if notification.channel == 'cache_invalidate':
            redis_client.delete(payload.get('key'))
        elif notification.channel == 'order_updates':
            websocket_broadcast('orders', payload)
```

Node.js (pg library)

```
const { Client } = require('pg');

const client = new Client({ connectionString: process.env.DATABASE_URL });
await client.connect();

client.on('notification', (msg) => {
    const payload = JSON.parse(msg.payload);
    console.log(`Channel: ${msg.channel}, Payload:`, payload);

    switch (msg.channel) {
        case 'order_updates':
            io.to(`order_${payload.order_id}`).emit('status_changed', payload);
            break;
        case 'cache_invalidate':
            cache.del(payload.key);
            break;
    }
});

await client.query('LISTEN order_updates');
await client.query('LISTEN cache_invalidate');
```

Elixir / Ecto (Phoenix PubSub)

```
# In Phoenix, Postgrex.Notifications handles LISTEN
{:ok, pid} = Postgrex.Notifications.start_link(database: "mydb")
{:ok, ref} = Postgrex.Notifications.listen(pid, "order_updates")
```

```
receive do
  {:notification, ^pid, ^ref, "order_updates", payload} ->
    Phoenix.PubSub.broadcast(MyApp.PubSub, "orders", {:order_changed, payload})
end
```

Production Monitoring Queries

```
-- List all active LISTEN connections and their channels
SELECT
    sa.pid,
    sa.usename,
    sa.application_name,
    sa.client_addr,
    sa.state,
    STRING_AGG(pc.channel, ', ') AS listening_channels
FROM pg_stat_activity sa
JOIN (
    -- pg doesn't expose listening channels per connection directly
    -- This is a known limitation; use application-level tracking
    SELECT 1 AS dummy
) sub ON true
WHERE sa.pid != pg_backend_pid()
GROUP BY sa.pid, sa.usename, sa.application_name, sa.client_addr, sa.state;

-- Count connections per state (listeners tend to be idle-in-transaction or idle)
SELECT state, COUNT(*)
FROM pg_stat_activity
GROUP BY state
ORDER BY COUNT(*) DESC;

-- Check pg_notification queue size (if notifications are piling up)
-- Not directly queryable, but monitor with:
SELECT pg_notification_queue_usage();
-- Returns fraction (0.0 to 1.0) of notification queue used
-- Alert if > 0.5 (50% full)

-- Find long-running LISTEN connections that may not be consuming
SELECT
    pid,
    usename,
    application_name,
    now() - state_change AS idle_time,
    state
FROM pg_stat_activity
WHERE state = 'idle'
  AND now() - state_change > interval '1 hour'
  AND application_name LIKE '%listener%';

-- Monitor notification delivery (application-level, not PostgreSQL built-in)
-- Create your own tracking table:
```

```
CREATE TABLE notification_log (
    id          BIGSERIAL PRIMARY KEY,
    channel     TEXT,
    payload     TEXT,
    sent_at     TIMESTAMPTZ DEFAULT now(),
    sender_pid  INT DEFAULT pg_backend_pid()
);

CREATE OR REPLACE FUNCTION logged_notify(p_channel TEXT, p_payload TEXT DEFAULT '')
RETURNS void LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO notification_log (channel, payload) VALUES (p_channel, p_payload);
    PERFORM pg_notify(p_channel, p_payload);
END;
$$;
```

Limitations and Gotchas

1. **No guaranteed delivery** — If the listener disconnects or is busy, notifications are lost. LISTEN/NOTIFY is fire-and-forget. For reliable messaging, use a proper queue (PGQ, PGMQ, or Kafka).
2. **Notification queue overflow** — If listeners don't consume fast enough, the in-memory notification queue can overflow (tracked by `pg_notification_queue_usage()`). On overflow, the oldest undelivered notifications are dropped.
3. **Not persistent** — Notifications are not written to disk. If PostgreSQL restarts, all pending notifications are lost.
4. **Within-connection delivery only** — A connection must call `pg_backend_pid()` / `conn.poll()` to receive notifications. There's no push; the application must actively check.
5. **NOTIFY in transactions** — Notifications are held in a pending buffer until `COMMIT`. A long-running transaction delays all its notifications.
6. **Channel names are case-insensitive** — `LISTEN OrderUpdates` is the same as `LISTEN orderupdates`. Use lowercase conventionally.
7. **Max payload: 8000 bytes** — Exceeding this raises an error. Always compute `length(payload)` before calling `pg_notify()`.
8. **No filtering** — All listeners on a channel receive all notifications. There's no subscriber-level filtering.

Best Practices

1. **Use IDs in payloads, not full data** — Send `{"id": 42}` and let the consumer query for fresh data. Reduces payload size and ensures the consumer always gets current data.
2. **Always measure `pg_notification_queue_usage()`** in production and alert at 50%.
3. **Use a persistent fallback** — For critical events, also write to a `pending_events` table. The consumer processes NOTIFY for real-time, but can recover missed events from the table.

4. **Prefix channel names** — Use `app_name.event_type` convention: `myapp.order_status`, `myapp.cache_invalidate`.

5. **Don't use LISTEN/NOTIFY as your only messaging system** — It's appropriate for low-volume, non-critical events (cache invalidation, dashboard refresh). Use Kafka/RabbitMQ for high-volume, reliable messaging.

Interview Questions & Answers

Q1: How does LISTEN/NOTIFY differ from polling for changes?

A: Polling requires the application to periodically query the database (`SELECT ... WHERE updated_at > last_check`), consuming CPU and database resources even when nothing has changed. LISTEN/NOTIFY is event-driven — the listener connection is dormant and consumes no resources until a notification arrives. NOTIFY is delivered after the triggering transaction commits, so latency is milliseconds rather than the polling interval (which might be seconds or minutes).

Q2: What happens to a NOTIFY if the transaction is rolled back?

A: The notification is discarded. PostgreSQL only delivers NOTIFY messages from successfully committed transactions. This is a key feature — it ensures listeners only receive notifications for changes that actually persisted. A rolled-back transaction's notification is never seen by any listener.

Q3: Explain the notification queue and what happens when it fills up.

A: PostgreSQL maintains an in-memory queue for pending notifications. When the queue approaches its limit (`pg_notification_queue_usage()` > 0.5), slow consumers risk missing notifications. If the queue fills completely, the oldest undelivered notifications are dropped — no error is raised. Monitor queue usage and use a persistent fallback table for critical events.

Q4: Can LISTEN/NOTIFY be used across databases?

A: No. NOTIFY delivers notifications only to listeners in the same database. Connections to different databases, even on the same PostgreSQL server, cannot receive each other's notifications. For cross-database event distribution, use logical replication, Debezium, or an external message broker.

Q5: Why must a LISTEN connection use autocommit mode?

A: Because LISTEN and UNLISTEN are not transaction-safe in the normal sense — they take effect at the session level, not the transaction level. When using libraries like psycopg2 in Python, you must set `isolation_level = 0` (autocommit). If you LISTEN inside a transaction and the transaction is long-running, you may miss notifications. Dedicated listener connections should always be in autocommit mode.

Q6: What is `pg_notification_queue_usage()` and how do you use it?

A: `pg_notification_queue_usage()` returns a float (0.0-1.0) representing what fraction of the async notification queue is currently used. Alert when it exceeds 0.5 (50%). It's not a per-channel metric — it shows total queue fullness across all channels and databases. Monitor it as a health metric for your LISTEN/NOTIFY infrastructure.

Q7: Describe a worker queue pattern using LISTEN/NOTIFY.

A: The job producer inserts into a `job_queue` table and a trigger fires `pg_notify('new_job', '...')`. The worker process holds a persistent connection with `LISTEN new_job`. When the server is idle, the worker

blocks efficiently (using `select()` or `conn.poll()` with timeout). On notification, it does `SELECT ... FOR UPDATE SKIP LOCKED LIMIT 1` to claim a job. This is more efficient than polling because the worker sleeps between jobs rather than querying continuously.

Q8: What are the alternatives to LISTEN/NOTIFY for reliable event processing?

A: (1) Dedicated queue tables with `SELECT ... FOR UPDATE SKIP LOCKED` (reliable but polling-based). (2) PGQ (skQueue extension) — a proper transactional queue built on PostgreSQL. (3) PGMQ (pgmq extension) — lightweight message queue for PostgreSQL. (4) Logical replication + Debezium + Kafka for distributed, high-volume events. (5) External brokers (Redis Streams, RabbitMQ, Kafka). Use LISTEN/NOTIFY for low-volume, non-critical real-time hints; use one of the above for reliable, high-volume messaging.

Exercises and Solutions

Exercise 1

Create an `orders` table with a trigger that sends a NOTIFY on every status change. In psql, use two sessions to demonstrate notification delivery.

Solution:

```
-- Setup
CREATE TABLE IF NOT EXISTS orders (
    id     SERIAL PRIMARY KEY,
    status TEXT DEFAULT 'pending'
);

CREATE OR REPLACE FUNCTION notify_order_status()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF OLD.status IS DISTINCT FROM NEW.status THEN
        PERFORM pg_notify('order_status',
            format('{"id":%s,"from":"%s","to":"%s"}',
                   NEW.id, OLD.status, NEW.status));
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_order_status_change
    AFTER UPDATE OF status ON orders
    FOR EACH ROW EXECUTE FUNCTION notify_order_status();

INSERT INTO orders (status) VALUES ('pending');  -- id=1

-- Session 1:  LISTEN order_status;
-- Session 2:  UPDATE orders SET status='shipped' WHERE id=1;
-- Session 1 receives: Asynchronous notification "order_status" with payload
--   '{"id":1,"from":"pending","to":"shipped"}'
```

Exercise 2

Write a function `safe_notify(channel TEXT, payload TEXT)` that checks payload length, truncates to the key fields if too long, and also writes to a `notification_audit` table.

Solution:

```
CREATE TABLE IF NOT EXISTS notification_audit (
    id         BIGSERIAL PRIMARY KEY,
    channel    TEXT,
    payload    TEXT,
    truncated  BOOLEAN DEFAULT false,
    sent_at    TIMESTAMPTZ DEFAULT now()
);

CREATE OR REPLACE FUNCTION safe_notify(p_channel TEXT, p_payload TEXT)
RETURNS void LANGUAGE plpgsql AS $$
DECLARE
    v_payload   TEXT := p_payload;
    v_truncated BOOLEAN := false;
BEGIN
    IF LENGTH(v_payload) > 7500 THEN
        v_payload := LEFT(v_payload, 7500) || '...TRUNCATED';
        v_truncated := true;
        RAISE WARNING 'Notification payload truncated for channel %', p_channel;
    END IF;

    INSERT INTO notification_audit (channel, payload, truncated)
    VALUES (p_channel, v_payload, v_truncated);

    PERFORM pg_notify(p_channel, v_payload);
END;
$$;

-- Test
SELECT safe_notify('test_channel', '{"msg": "hello"}');
SELECT safe_notify('big_channel', REPEAT('x', 8000));  -- triggers truncation

SELECT * FROM notification_audit ORDER BY sent_at DESC LIMIT 5;
```

Cross-References

- **09_stored_procedures_functions.md** — Trigger implementations for NOTIFY
- **06_logical_decoding.md** — Alternative for reliable change streams (NOTIFY is not reliable)
- **04_materialized_views.md** — NOTIFY for triggering matview refreshes
- **17_postgresql_for_backend_engineers** — WebSocket integration patterns
- **PostgreSQL Docs** — https://www.postgresql.org/docs/current/sql-notify.html